

Password Hashing & Identity Bootstrap

PBKDF2 / BCrypt hashing
mechanics · TmsUser · IdentityDbContext · AuthController Register & Login

Identity bootstrap

```
builder.Services.AddIdentityCore<TmsUser>(  
    options => {  
        options.Password.RequiredLength = 12;  
    });
```

Password Hashing Mechanics & ASP.NET Core Identity

1. The Big Picture: Plain-Text Vulnerability vs Hashing Mechanics

- The Plain-Text Leak Risk: If user passwords are stored in plain text or using outdated fast algorithms (MD5, single SHA256), a database leak compromises all user credentials instantly across platforms.

CryptoDemoService.cs

```
// BCrypt.Net-Next (Exercise 1 Hashing Mechanics)

string salt = BCrypt.Net.BCrypt.GenerateSalt(12);

string hashedPassword = BCrypt.Net.BCrypt.HashPassword(rawPassword, salt);

bool isValid = BCrypt.Net.BCrypt.Verify(rawPassword, hashedPassword);
```

- Key Takeaway: Passwords must NEVER be stored or encrypted — they must be one-way salted and hashed.

2. Authentication vs Authorization

Concept	Question	Mechanism	Server Response
AuthN (Authentication)	Who are you?	Credentials check, Identity verification	401 Unauthorized
AuthZ (Authorization)	What can you do?	Role & Policy evaluation	403 Forbidden

- Note: In HTTP status code standards, 401 literally means 'Unauthenticated', whereas 403 means 'Forbidden' (Authenticated, but insufficient privileges).

3. ASP.NET Core Identity Configuration

- We integrate ASP.NET Core Identity with custom domain user properties (TmsUser extending IdentityUser) and EF Core persistence (IdentityDbContext<TmsUser>).

```
// Program.cs
builder.Services.AddIdentityCore<TmsUser>(options =>
{
    options.Password.RequiredLength = 12;
    options.Password.RequireUppercase = true;
    options.Password.RequireDigit = true;

    options.Password.RequireNonAlphanumeric = true;

    options.Lockout.MaxFailedAccessAttempts = 5;

    options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);

    options.Lockout.AllowedForNewUsers = true;

})

.AddRoles<IdentityRole>()

.AddEntityFrameworkStores<TmsDbContext>();
```

4. AuthController: Register & Login Flows

```
[public class AuthController : ControllerBase {

    private readonly UserManager<TmsUser> _userManager;

    [HttpPost("register")]
    public async Task<IActionResult> Register(RegisterDto dto) {
        var user = new TmsUser { UserName = dto.Email, Email = dto.Email, FirstName = dto.FirstName, LastName = dto.LastName };

        var result = await _userManager.CreateAsync(user, dto.Password);

        if (!result.Succeeded) return BadRequest(result.Errors);

        await _userManager.AddToRoleAsync(user, dto.Role ?? "Student");

        return StatusCode(201, new { message = "User registered successfully." });
    }

    [HttpPost("login")]
    public async Task<IActionResult> Login(LoginDto dto) {
        var user = await _userManager.FindByEmailAsync(dto.Email);
        if (user == null) return Unauthorized(new { message = "Invalid email or password." });
        if (await _userManager.IsLockedOutAsync(user)) return StatusCode(423, new { message = "Account locked out..." });
        var valid = await _userManager.CheckPasswordAsync(user, dto.Password);

        if (!valid) { await _userManager.AccessFailedAsync(user); return Unauthorized(...); }

        await _userManager.ResetAccessFailedCountAsync(user);

        return Ok(new { message = "Authentication successful." });
    }
}
```

5. Session 1 Traps & Common Mistakes

Mistake	Symptom	Enterprise Fix
Email Enumeration	API returns "Email not found"	Return generic "Invalid email or password"
Bypassing UserManager	Adding TmsUser directly to DbContext	Always use _userManager.CreateAsync() for hashing
Missing EF Migration	Database error on login/register	Run dotnet ef migrations add AddIdentitySupport

Next: M11 Session 2 — JWT Bearer Authentication & Refresh Token Rotation

Token generation (TokenService), claims, AddJwtBearer pipeline, and Refresh Token Rotation.